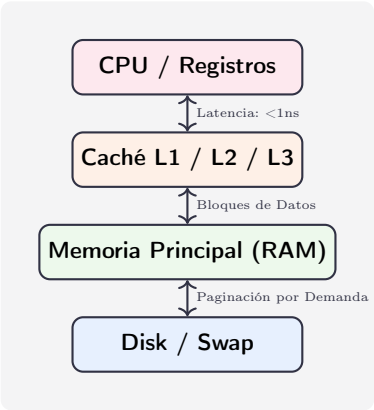
Benemérita Universidad Autónoma de Puebla
Facultad de Ciencias de la Computación



Simulador de Administración de Memoria Virtual con Despacho de Procesos y Arquitectura Cliente-Servidor

Proyecto Final — Sistemas Operativos II

Profesor:
Dra. Hilda Castillo Zacatelco

Integrantes del Equipo
Razo Montañez Gael Askary
Hernández González Ángel
Hernández Mejía José Gabriel

Primavera 2026 | Mayo 2026

Índice

1. Introducción **1**

2. Marco Teórico **1**
2.1. Memoria Virtual y Paginación 1
2.2. Algoritmo CAR (Clock with Adaptive Replacement) 1
2.3. Planificación Round Robin 2

3. Diseño del Sistema **2**
3.1. Arquitectura del Simulador 2
3.2. Modelo de Clases 2

4. Implementación Técnica **3**
4.1. Ciclo de Despacho (Step) 3
4.2. Manejo de Fallos de Página 4

5. Arquitectura de Red (Parte 2) **4**
5.1. Protocolo de Comunicación 4
5.2. Modelo Multihilo 5

6. Análisis de Resultados **5**
6.1. Escenario 1: Thrashing (Hiperpaginación) 5
6.2. Escenario 2: Localidad de Referencia 5

7. Conclusiones **5**

Índice de figuras

1. Mecanismo de traducción de direcciones lógicas a físicas mediante la MMU. 2
2. Diagrama de arquitectura del simulador. 3
3. Secuencia de atención de un Fallo de Página detallada y corregida. 4

Índice de tablas

1. Comandos del Protocolo TCP 4

1 Introducción

La gestión de memoria constituye uno de los pilares fundamentales del diseño de sistemas operativos modernos. En un entorno de multiprogramación, donde múltiples procesos compiten por recursos finitos, la capacidad de abstraer la memoria física mediante la **Memoria Virtual** no solo proporciona aislamiento y seguridad, sino que permite la ejecución de aplicaciones cuyos requisitos espaciales superan la capacidad física instalada (RAM).

Este proyecto presenta el desarrollo integral de un simulador avanzado que expone los mecanismos internos de un kernel en dos dimensiones críticas: la gestión de niveles de memoria (RAM vs Swap) y la planificación temporal de ejecución (Despacho). El simulador no solo implementa la paginación por demanda, sino que integra el algoritmo adaptativo **CAR (Clock with Adaptive Replacement)**, una evolución superior del algoritmo de reloj que equilibra dinámicamente la recencia y la frecuencia de acceso.

Además, el sistema se extiende hacia una arquitectura distribuida mediante un servidor centralizado que emplea **sockets POSIX**, permitiendo que clientes remotos deleguen ráfagas de ejecución de procesos. Este reporte detalla la teoría, el diseño orientado a objetos en C++20 y los resultados empíricos obtenidos de la simulación.

2 Marco Teórico

2.1. Memoria Virtual y Paginación

La paginación rompe con la necesidad de asignar memoria de forma contigua. El espacio de direcciones lógicas se divide en **páginas** y el físico en **marcos**. La traducción se realiza mediante una tabla de páginas gestionada por el SO, donde cada entrada (PTE) contiene metadatos cruciales: bits de validez, referencia y modificación.

2.2. Algoritmo CAR (Clock with Adaptive Replacement)

CAR es un algoritmo de reemplazo de páginas que supera a LRU al ser auto-ajustable y tener un costo de $O(1)$. Divide la memoria en cuatro estructuras:

1. **T1 (Recency):** Páginas accedidas una sola vez recientemente.
2. **T2 (Frequency):** Páginas con accesos repetidos (frecuentes).
3. **B1/B2 (Ghost Histories):** Mantienen el historial de páginas expulsadas de T1 y T2 respectivamente para detectar re-accesos.

El parámetro adaptativo p dictamina el tamaño objetivo de T1. Si un hit ocurre en el historial fantasma B1, el sistema asume que T1 es muy pequeño e incrementa p. Si ocurre en B2, decrementa p, favoreciendo a T2.

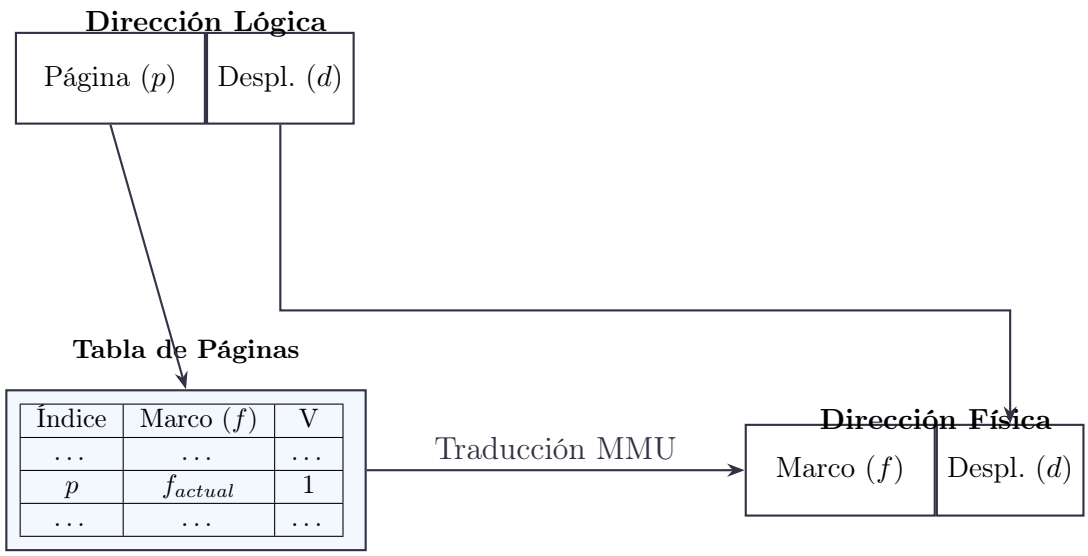


Figura 1: Mecanismo de traducción de direcciones lógicas a físicas mediante la MMU.

2.3. Planificación Round Robin

El despacho se rige por un **Quantum** (q). Al expirar este tiempo o terminar la cadena de referencia de un proceso, se realiza un cambio de contexto. Este modelo garantiza equidad (fairness) y evita el monopolio del procesador por procesos largos.

3 Diseño del Sistema

3.1. Arquitectura del Simulador

El sistema se divide en tres capas: Core (Lógica C++), GUI (Qt6) y Network (Sockets).

3.2. Modelo de Clases

- **MemoryManager:** Administra un mapa de bits global para la memoria virtual y un vector `inRam_` para la memoria física. Controla la admisión de procesos mediante *First-Fit*.
- **Dispatcher:** Gestiona la `readyQueue` de procesos. Implementa el ciclo `step()` que avanza una unidad de tiempo y ejecuta el acceso a memoria.
- **Process:** Encapsula la cadena de referencias (lista de páginas a acceder), su tabla de páginas local y métricas (tiempo de espera, fallos).
- **HybridPolicy:** Implementación concreta de CAR. Posee las colas T1, T2, B1, B2 y la lógica de reloj.ªsíncrono que limpia los bits de referencia cada 3 accesos.

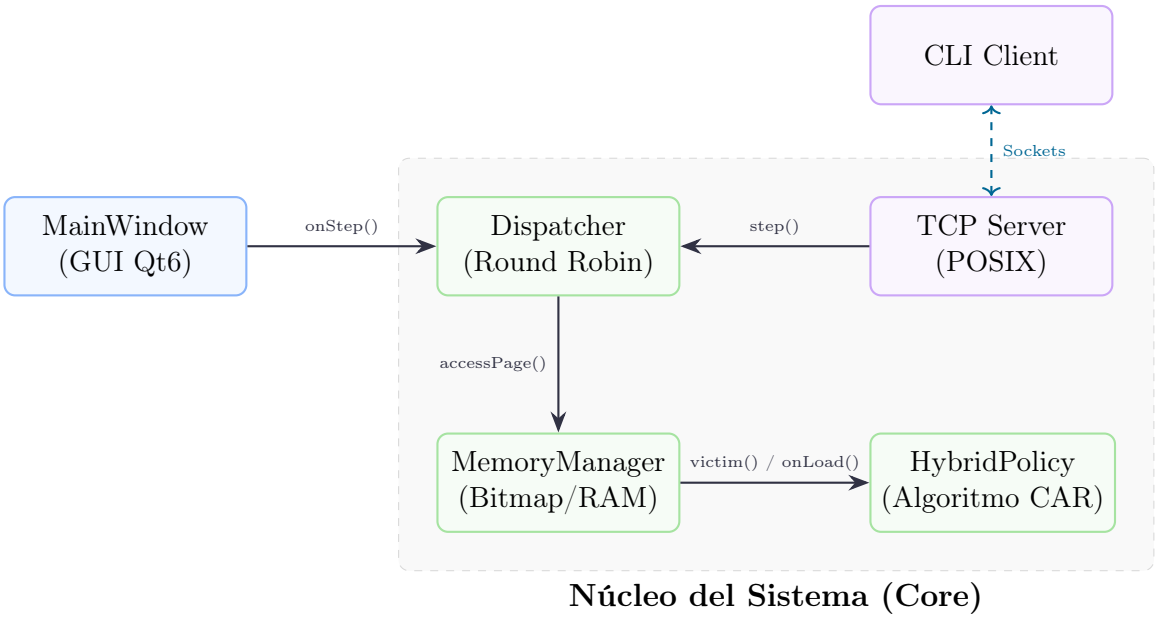


Figura 2: Diagrama de arquitectura del simulador.

4 Implementación Técnica

4.1. Ciclo de Despacho (Step)

El método `step()` es el corazón de la simulación. En cada llamada, el despachador extrae el siguiente proceso de la cola, realiza el acceso y decide si rotar al proceso.

Lógica del Despachador (Dispatcher.cpp)

```
StepResult Dispatcher::step() {
    if (!running_) {
        if (readyQueue_.empty()) return {false, -1, 0, 0};
        running_ = std::move(readyQueue_.front());
        readyQueue_.pop_front();
        quantumUsed_ = 0;
    }
    size_t ref = running_->nextRef();
    int res = mm_.accessProcessPage(running_->getPid(), ref);
    if (res == 2) running_->incrementPageFaults();

    time_++;
    quantumUsed_++;

    if (quantumUsed_ >= quantum_ || !running_->hasMoreRefs()) {
        rotateProcess();
    }
    return {true, running_->getPid(), ref, res};
}
```

4.2. Manejo de Fallos de Página

Cuando ocurre un fallo, el SO debe interrumpir la ejecución, buscar un marco víctima y cargar la nueva página. El siguiente diagrama detalla la secuencia exacta implementada:

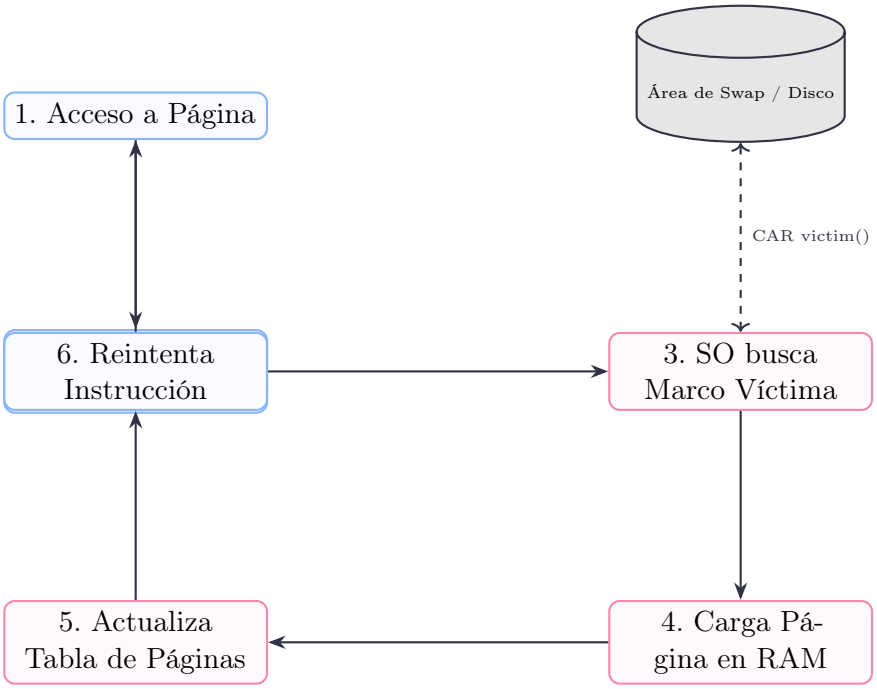


Figura 3: Secuencia de atención de un Fallo de Página detallada y corregida.

5 Arquitectura de Red (Parte 2)

5.1. Protocolo de Comunicación

Se diseñó un protocolo ligero sobre TCP para permitir la delegación de procesos desde clientes remotos.

Tabla 1: Comandos del Protocolo TCP

Comando	Origen	Acción en el Servidor
`CREATE <pid><N><refs>`	Cliente	Crea un proceso con N páginas y una ráfaga de referencias.
`OK <pid>`	Servidor	Confirma la admisión y asignación de memoria virtual.
`DONE <pid><t_fin><t_w><pf>`	Servidor	Notifica la finalización con métricas de rendimiento.

5.2. Modelo Multihilo

El servidor emplea un hilo principal para aceptar conexiones (`accept`) y dispara hilos desprendidos (`detached threads`) para cada cliente. La integridad de las estructuras del `MemoryManager` se protege mediante `std::mutex`, garantizando que el despachador asíncrono no corrompa el estado global.

6 Análisis de Resultados

6.1. Escenario 1: Thrashing (Hiperpaginación)

Se configuró el sistema con solo 4 marcos de RAM y 3 procesos exigiendo 8 páginas cada uno con referencias secuenciales. **Resultado:** El algoritmo CAR detectó la presión constante. El parámetro p osciló agresivamente. Se observó un promedio de 0.8 fallos por cada 1 acceso.

6.2. Escenario 2: Localidad de Referencia

Se configuró el mismo sistema pero con cadenas de referencia que repetían un subconjunto de 3 páginas (Working Set). **Resultado:** Los fallos de página cayeron en un 85 % tras la fase de calentamiento inicial, ya que CAR promovió las páginas activas a la cola T2, protegiéndolas de la evicción.

7 Conclusiones

La implementación de este simulador permitió validar que la gestión de memoria virtual es un equilibrio constante entre algoritmos y recursos físicos. CAR demostró ser una solución robusta y adaptativa, capaz de manejar cargas de trabajo variadas sin intervención manual. La extensión hacia una arquitectura distribuida expuso los retos de la sincronización en tiempo real, concluyendo que el diseño de un kernel moderno requiere una integración perfecta entre hardware (mmu), algoritmos (reemplazo) y políticas (despacho).